Project Portfolio: Snehal Nair

Machine Learning & AI Systems Engineering

Project 1: Cost-Aware Automatic Prompt Optimization (APO)

Situation: LLMs are highly sensitive to prompt phrasing, but prompt engineering is often manual and difficult to scale. At Viator, I transitioned from a multi-label taxonomy to a single-label, multiclass taxonomy to improve product discoverability. Classifying products into 66 exclusive categories with limited labelled data across diverse destinations was challenging, and existing APO frameworks (APE, OPRO, ProTeGi) focused on simple binary tasks without considering the cost-performance trade-offs required in commercial settings.

Task: Automate optimal prompt generation for product classification, maximizing Weighted F1 while minimizing inference costs. This required evaluating existing APO approaches and engineering a cost-efficient solution suitable for thousands of classifications.

Action: Built a comprehensive evaluation framework with my team and created a new hybrid architecture:

1. Hybrid APE-OPRO Architecture
Used APE's semantically diverse prompt generation (temperature=1.0, 20 candidates) to initialize OPRO, avoiding OPRO's costly cold-start iterations. APE provided strong initial prompts, while OPRO refined them through iterative optimization.

2. Optimizer-Scorer Loop with Cost Controls

- Optimizer Model: GPT-4 for candidate prompt generation (high reasoning capability)
- Scorer Model: GPT-3.5-Turbo for evaluation (10x cost reduction vs. GPT-4)
- Stratified Sampling: Ensured rare categories (e.g., 'Submarine Tours') appeared in evaluation batches proportional to commercial importance
- Evaluation Budget: 200 samples per iteration with 3-fold validation to reduce variance

3. Ablation Studies
Systematically tuned breadth (prompt count: 5, 10, 20) and depth (iterations: 3, 5, 10) to identify diminishing returns. Found optimal configuration at 10 prompts × 5 iterations.

Results

Metric	Manual Baseline	OPRO Only	Hybrid APE-OPRO
Weighted F1	0.78	0.84	0.84
API Cost (per optimization run)	$45	$52	$42.60
Optimization Time	N/A	4.2 hours	2.8 hours

Prompt Stability (across 3 model versions)	N/A	62%	78%

The hybrid method matched state-of-the-art performance while reducing API costs by 18% compared to OPRO alone and 5% compared to manual iteration. Cost reduction calculated as: (OPRO cost - Hybrid cost) / OPRO cost = ($52 - $42.60) / $52 = 18.1%.

System Design & Architecture

Architecture Overview: Modular two-phase system with decoupled optimization and evaluation:

- Phase 1 (Expansion): APE generates 20 diverse candidate prompts using high-temperature sampling
- Phase 2 (Scoring): Miner-Critic loop evaluates candidates, feeding top-3 prompts back into optimizer for refinement
- Termination: Early stopping when F1 improvement < 0.5% for 2 consecutive iterations

Prompt Stability & Model Version Handling: Optimized prompts often break when underlying LLMs are updated. Implemented:

- Version Regression Testing: Automated pipeline re-evaluates top prompts against new model versions within 24 hours of release
- Prompt Ensemble: Maintain top-3 prompts; if primary degrades >5% F1, automatically failover to secondary
- Semantic Anchoring: Prompts include explicit format constraints that are less sensitive to model changes

Evaluation Rigor: LLM-as-judge has high variance. Mitigated through:

- Multi-Evaluation: Each prompt evaluated 3 times; final score is median to reduce outlier impact
- Confidence Intervals: Report F1 as mean ± std (e.g., 0.84 ± 0.02) across evaluation runs
- Human Calibration: Quarterly audit of 100 random classifications against human labels; correlation >0.92

Risks & Mitigations

Risk	Impact	Mitigation	Monitoring
Cost Explosion	OPRO generates long prompts	APE initialization flattens cost curve; token budget cap at 500 tokens/prompt	Real-time cost dashboard with alerts at 80% budget
Label Sensitivity	Performance varies with label format	Standardized templates; ablation on 5 format variants	A/B test new formats before deployment

Project 2: Real-Time Personalized Search Ranking

Situation: The legacy search system relied on popularity and keyword matching, failing to adapt to real-time user intent. Users exploring niche categories continued to see globally popular but irrelevant products, creating relevance gaps and lowering conversions.

Task: Build Ranker V2: a personalized Learning-to-Rank system using recent user behavior, with strict latency (<50ms P95) and measurable relevance improvement (NDCG@10). Required robust MLOps to avoid training-serving skew.

Action: Two-Stage Hybrid Architecture
Stage 1: Hybrid Retrieval Layer

- **Lexical**: Solr with BM25 scoring for exact keyword matches
- **Semantic**: Qdrant with all-MiniLM-L6-v2 embeddings (384 dimensions, chosen for latency/quality tradeoff after benchmarking against MPNet and E5)
- **Query Understanding**: spaCy NER pipeline extracts location entities; hard geographic filters prevent Sydney tours appearing for 'Paris wine tasting'
- **Candidate Set**: Top 100 from each retriever, deduplicated, passed to re-ranker

Stage 2: Re-Ranker (Ensemble Architecture)
LightGBM and DCN serve complementary roles in an ensemble:

- LightGBM (Primary): Handles sparse categorical features (product category, supplier tier, user segment) with 800 trees, max_depth=8
- DCN-v2 (Secondary): Captures dense feature interactions between user embeddings and product embeddings via cross-network layers
- Ensemble: Weighted combination (0.7 LightGBM + 0.3 DCN) based on offline ablation showing LightGBM alone at 3.1% NDCG lift vs. ensemble at 4.2%

Session Personalization: Window selection based on user behavior analysis. The 3-hour window optimizes coverage vs. freshness. Shorter windows miss multi-session planners; longer windows introduce stale intent signals.

Window	Session Coverage	Signal Freshness	NDCG@10 Lift
1 hour	34%	High	+2.8%
3 hours	67%	Medium-High	+4.2%
6 hours	78%	Medium	+3.9%
24 hours	89%	Low	+2.1%

Position Bias Correction: Click data is heavily biased toward top-ranked items. Implemented:

- Inverse Propensity Scoring (IPS): Estimated position bias from randomization experiments; down-weighted clicks on positions 1-3 by 0.4x

- Trust Bias Model: Separate model predicts P(click|position) vs. P(click|relevance); used to debias training labels
- Exploration Budget: 5% of traffic sees randomized rankings to collect unbiased feedback

Cold-Start Handling: 67% of sessions have no prior clicks in the 3-hour window. Fallback strategy:

- Tier 1 (No clicks): Use entry point signals (landing page, referrer, device) + location-based popularity
- Tier 2 (1-2 clicks): Partial personalization with high uncertainty penalty
- Tier 3 (3+ clicks): Full personalization model

MLOps & Data Integrity

- Dual Pipelines: Separate training and serving feature stores with hourly sync validation
- Causal Filtering: Training excludes future information (no post-click signals at inference time)
- Data Quality Gates: Training halts if >2% feature nulls or >5% label noise detected

Results

Metric	Baseline	Ranker V2	Improvement
NDCG@10 (Offline)	0.412	0.429	+4.1%
NDCG@10 (Online A/B)	—	—	+3.8% ($p<0.01$)
Conversion Rate	2.1%	2.3%	+9.5%
P95 Latency	68ms	47ms	-31%
Cold-Start NDCG@10	0.398	0.415	+4.3%

Online A/B test ran for 3 weeks with 50/50 traffic split (n=2.4M sessions). Offline-online gap of 0.3% attributable to position bias in live traffic.

System Design & Architecture

Component	Technology	Rationale
MLOps	Valohai	Reproducible training pipelines with experiment tracking
Serving	Seldon on Kubernetes	Auto-scaling, A/B routing, shadow mode for new models
Vector Store	Qdrant (primary), FAISS (fallback)	Qdrant for filtering; FAISS for pure speed
Feature Store	Redis (online), Parquet (offline)	Sub-ms feature lookup; consistent training/serving
Monitoring	Prometheus + Grafana	Real-time latency, throughput, model metrics

Risks & Mitigations

Risk	Impact	Mitigation	Monitoring
Position Bias	Model reinforces existing rankings	IPS weighting + 5% exploration traffic	Compare explore vs. exploit NDCG weekly
Feedback Loops	Rich-get-richer dynamics	Diversity injection: ensure 20% results from tail	Gini coefficient of impression distribution
Training-Serving Skew	Offline gains don't translate online	Feature parity validation before deployment	Daily skew detection comparing feature distributions
Latency Creep	Heavy features slow inference	Pre-computed embeddings, vectorized ops, caching	P95/P99 latency alerts at 45ms/60ms
Cold Start	Poor experience for new users	Tiered fallback strategy + location priors	Segment NDCG for 0-click vs. 3+ click users

Project 3: Review Summarization at Scale

Situation: LLM summarizers struggled with thousands of redundant reviews, producing hallucinations or overly positive summaries. Capturing nuanced customer sentiment required a more grounded, scalable approach.

Task: Build a domain-agnostic pipeline that generates explainable, grounded summaries by extracting structured information and removing redundancy, with explicit faithfulness guarantees.

Action
Phase 1: Hierarchical Theme Identification

- Zero-Shot Generation: GPT-4 generates candidate themes from a sample of 500 reviews
- Theme Validation: Human annotators label 200 reviews against generated themes; themes with <70% agreement discarded
- Semantic Deduplication: all-mpnet-base-v2 embeddings (768 dim) with cosine similarity >0.85 threshold to merge overlapping themes
- Theme Coverage Metric: % of reviews where at least one theme applies (validated against human labels)

Phase 2: Structured Extraction (ABSA)

Aspect-Based Sentiment Analysis extracts (Theme, Aspect, Opinion, Sentiment) tuples:

- Primary Extractor: Fine-tuned DeBERTa-v3-base on 5,000 labeled review sentences
- Secondary Validator: GPT-3.5 validates extractions; inter-model agreement: 87.3%
- Conflict Resolution: When models disagree, sample sent to human review queue

Phase 3: Opinion Clustering (Key Innovation)

- Embedding Model: all-MiniLM-L6-v2 (selected for speed; benchmarked against mpnet with <2% quality loss)
- Clustering: HDBSCAN with min_cluster_size=5, min_samples=3 (tuned via silhouette score)
- Representative Selection: Cluster medoid + 2 diverse samples per cluster
- Reduction: 5,000 reviews → ~150 representative opinions

Multilingual Handling

Viator operates globally; 34% of reviews are non-English:

- Language Detection: fastText lid.176 model
- Translation: NLLB-200 for non-English reviews before processing
- Validation: Native speaker spot-checks for top-5 languages (ES, FR, DE, IT, PT)

Results

Metric	Baseline (Direct LLM)	Pipeline	Method
Theme Coverage	~50%	94.2%	Human annotation on 500 review sample
Sentiment Accuracy	71%	89%	Compared to 3-annotator majority vote
Positivity Bias	82% positive	67% positive	Ground truth distribution: 65% positive
Hallucination Rate	12%	1.8%	Manual audit of 200 summaries
Token Usage (per product)	45K	8K	82% reduction via clustering

System Design & Architecture

Hybrid ABSA + clustering + LLM summarization pipeline:

- Input: Raw reviews from product database
- Stage 1: Theme identification and validation
- Stage 2: ABSA extraction with dual-model validation
- Stage 3: Opinion clustering and representative selection
- Stage 4: LLM generates summary from representatives only (grounded)
- Output: Structured summary with sentiment distribution and source traceability

Sentiment Calibration Methodology: To validate 'removed positivity bias' claim:

1. Collected ground truth sentiment distribution from 1,000 manually labeled reviews
2. Baseline LLM summaries implied 82% positive sentiment (overestimate)
3. Pipeline output: 67% positive, within 2% of ground truth (65%)
4. Statistical test: Chi-square $p<0.01$ for baseline vs. ground truth; $p=0.34$ for pipeline vs. ground truth

Risks & Mitigations

Risk	Impact	Mitigation	Monitoring
Hallucination	Summaries contain invented details	Faithfulness check: every claim traced to source review	Weekly audit of 50 random summaries
Theme Overlap	Redundant or confusing themes	Dual-level frequency analysis; merge threshold tuning	Theme drift detection monthly
Embedding Quality	Poor clustering affects representatives	Benchmark 3 embedding models; select best silhouette	Cluster quality metrics in dashboard
Translation Errors	Non-English reviews misprocessed	NLLB + native speaker validation for top languages	BLEU score monitoring for translation quality
Cost/Input Size	Thousands of reviews per product	Clustering reduces to ~150 representatives	Token usage alerts per product

Project 4: Automated FAQ Extraction (Knowledge Governance System)

Situation: At Viator, the 'industry standard' approach of clustering chat logs and summarizing answers failed in production. A pilot revealed that models would hallucinate questions to fit positive sentiment (e.g., claiming food was included because a chat mentioned pets). Because customers are 'chaotic browsers' who often chat about one product while viewing another, 30% of support data was contaminated. This led to staleness, inconsistency, and poor coverage for long-tail products.

Task: Reframe from a simple prompting problem to a system design problem. Build a 'Knowledge Lifecycle' system that keeps FAQs true over time and at scale. Targets: 90% reduction in hallucinations and 30% reduction in informational support tickets.

Action

Pillar 1: Self-Adaptive RAG (Five-Stage Pipeline)

1. Query Analysis: Classify incoming query by intent type (policy, product-specific, general)

2. Source Selection: Route to appropriate knowledge base (policies, product data, chat history)
3. Retrieval: Hybrid semantic + lexical search with MMR diversification
4. Source Authority Weighting: Score sources by tier—Gold (official policies, weight=1.0), Silver (structured product data, weight=0.7), Bronze (chat logs, weight=0.3)
5. Self-Critique: Retrieved context evaluated for relevance and sufficiency; re-retrieve if confidence <0.8

Pillar 2: GraphRAG Knowledge Structure
Ontology Schema:

- Entity Types: Product, Supplier, Policy, FAQ, Attribute
- Relationship Types: HAS_POLICY, SUPERSEDES, INHERITS_FROM, APPLIES_TO
- Schema Evolution: New entity/relationship types require PR review; backward-compatible migrations only

Truth Propagation Example:
When 'Weather Policy' node updates to allow cancellations for >30°C heat, all linked Product nodes' FAQs are flagged for regeneration. Graph traversal ensures no orphaned FAQs.

Pillar 3: Agentic Validation (Coordination Architecture)

Agent	Trigger	Action	Output
Monitor	Hourly cron + webhook on data change	Detect price/policy/API drift	Change event to queue
Validate	Change event received	Check FAQ accuracy against current data	Validation score + delta
Correct	Validation score <0.9 or delta >threshold	Trigger FAQ regeneration	New FAQ candidate
Govern	New FAQ candidate	Compare to previous; enforce quality gates	Publish or human review

Coordination: Redis Streams for event queue; agents are stateless workers consuming from streams. Idempotency keys prevent duplicate processing. Circuit breaker prevents cascade failures (max 100 regenerations/hour).

Results

Metric	Baseline	System	Method
Hallucination Rate	23%	2.1%	Manual audit of 500 FAQs

Ticket Deflection	—	34%	A/B test: FAQ-enabled vs. control
Product Coverage (Day 1)	12%	81%	Products with ≥3 FAQs
FAQ Freshness	45 days avg	< 24 hours	Time from source change to FAQ update
LLM Latency (serving)	2.1s	0ms	Cache hit rate 99.2%

Sibling Transfer Validation
When transferring FAQs from similar products:

1. Similarity threshold: Products must share >80% attribute overlap
2. Validation: Transferred FAQ passed through Validate agent with source product context
3. Confidence scoring: Transferred FAQs marked with confidence; <0.85 triggers human review
4. Audit: 10% random sample of transfers reviewed weekly; current accuracy: 91%

System Design & Architecture

Offline Generation / Online Serving Split

- Offline: FAQ generation runs async; no latency impact on user experience
- Online: FAQs served from Redis cache; cache miss falls back to Elasticsearch
- Consistency: Write-through cache; FAQ updates atomic with cache invalidation

Graph Update Consistency
Policy node changes trigger cascading regeneration:

- Transaction Model: Graph updates wrapped in Neo4j transactions; rollback on partial failure
- Batch Processing: Downstream regenerations queued and processed in batches of 50
- Progress Tracking: Each regeneration job tracked; incomplete jobs resumed on worker restart
- Consistency Check: Daily job verifies all FAQs are consistent with current graph state

Risks & Mitigations

Risk	Impact	Mitigation	Monitoring
Cascading Regeneration	Policy change triggers thousands of updates	Circuit breaker (100/hr max); priority queue for high-traffic products	Regeneration queue depth alerts
False Positive Floods	Minor price changes trigger unnecessary work	Dampening layer: ignore changes <5% or within normal variance	False positive rate tracking

Regression on Correction	New FAQ worse than stale original	Critic score comparison; auto-rollback if score drops >10%	Before/after score distribution
Entity Resolution	Sibling transfer fails on entity mismatch	Fuzzy matching + manual entity linking for top suppliers	Entity resolution accuracy dashboard
Graph Staleness	Reality changes faster than graph updates	Agentic monitoring loop; webhook integration with data sources	Graph freshness SLA monitoring

Project 5: Active Learning for Traveler Tips Extraction

Situation: Traveler tips, valuable, actionable advice, were buried within user reviews. Lack of labeled data made conventional model training impractical.

Task: Fine-tune a Small Language Model to extract tips with accuracy exceeding few-shot LLM baselines, while minimizing labeling cost and maintaining production-viable inference costs.

Action
Model Selection: Evaluated three SLM candidates.Selected DeBERTa-v3-small: Best balance of quality and latency for production serving.

Model	Parameters	Inference Latency	Base F1 (few-shot)
DistilBERT	66M	12ms	0.68
DeBERTa-v3-small	44M	18ms	0.71
Flan-T5-small	80M	25ms	0.74

Active Learning Workflow

1. **Initial Dataset**: 4,000 samples with 50% tips (balanced for training stability). 2,000 from LLM-generated labels (GPT-3.5), 2,000 from existing weak labels.
2. **Teacher Model**: GPT-4 as oracle for disagreement mining
3. **Disagreement Mining**: Ran inference on 10,000 unlabeled reviews; flagged 1,200 where student prediction diverged from teacher by >0.3 confidence
4. **Human Review**: Labeled 500 highest-disagreement samples (40 hours annotator time)
5. **Iteration**: Repeated cycle 3 times until F1 improvement <1% per cycle

Distribution Calibration
Training on 50/50 balanced data creates probability miscalibration in production (tips are ~8% of reviews). Mitigation:

- Temperature Scaling: Post-hoc calibration on held-out set with true distribution

- Threshold Tuning: Operating threshold set to 0.72 (vs. default 0.5) to optimize precision-recall tradeoff
- Production Sampling: Inference outputs calibrated probabilities, not raw logits

Teacher Bias Mitigation

LLM-generated labels introduce teacher model biases:

- Bias Detection: Compared LLM labels to human labels on 200 samples; identified systematic under-labeling of negative tips ('avoid...')
- Correction: Augmented training data with human-labeled negative tip examples
- Validation: Final model evaluated only on human-labeled test set (not on LLM labels)

Results

Metric	Few-Shot GPT-4	Fine-Tuned DeBERTa	Method
F1 Score	0.76	0.84	Human-labeled test set (n=500)
Precision	0.72	0.88	—
Recall	0.81	0.80	—
Inference Cost (per 1K reviews)	$4.20	$0.03	API vs. self-hosted
Latency (P95)	1.2s	22ms	—

Model achieves 10.5% F1 improvement over few-shot baseline while reducing inference cost by 99.3%.

System Design & Architecture

- Training: PyTorch + HuggingFace Transformers; 3 epochs, learning rate 2e-5, batch size 32
- Serving: ONNX-optimized model on CPU instances (cost-effective for this workload)
- Pipeline: Batch inference nightly on new reviews; results cached in Elasticsearch

Risks & Mitigations

Risk	Impact	Mitigation	Monitoring
Class Imbalance	Model biased toward majority class	Balanced training + threshold calibration	Per-class precision/recall weekly
Teacher Bias	LLM errors propagated to student	Human validation set; bias detection pipeline	Disagreement rate tracking
Diminishing Returns	Continued labeling wastes resources	Stop when F1 improvement <1% per cycle	Learning curve monitoring
Distribution Shift	Review language evolves over time	Quarterly re-evaluation on fresh samples	F1 trend monitoring monthly

Project 6: Enterprise AI Governance Framework (Responsible AI)

Situation: As Viator scaled its use of Large Language Models across multiple customer-facing domains, the lack of centralized oversight posed significant brand and safety risks. Disparate teams deployed prompts without standardized safety guardrails, leading to inconsistent outputs and potential exposure to adversarial inputs.

Task: Design and implement an organization-wide AI Governance and Responsible AI Framework to standardize safety, compliance, and ethical oversight. Goal: 100% of production models pass a rigorous Safety Gate without significantly increasing deployment latency.

Action

Pillar 1: Safety Gate Implementation ('Critic-at-the-Edge')

Architecture achieving <10ms validation:

- PII Detection: Regex patterns for common PII (email, phone, SSN) + Presidio for entity recognition (runs in parallel)
- Content Classification: DistilBERT fine-tuned on 10K labeled examples for prohibited content categories
- Bias Detection: Lightweight heuristics (word lists, sentiment skew) for initial screen; heavy model for flagged cases only
- Batching: Requests batched at 10ms intervals; amortizes model inference overhead
- Total Latency: P50: 4ms, P95: 8ms, P99: 12ms (measured over 1M requests)

Pillar 2: Adversarial Robustness

Safety layers vulnerable to bypass attempts. Implemented:

- Red Team Program: Quarterly adversarial testing by internal security team + external bug bounty
- Encoding Detection: Base64, ROT13, Unicode homoglyph detection in input preprocessing
- Multi-Turn Analysis: Conversation-level context tracking to detect jailbreak attempts across turns
- Canary Tokens: Synthetic 'honeypot' prompts in production to detect bypass attempts

Pillar 3: Source Authority Hierarchies

Prevents misinformation spread:

- Gold Sources: Official policy documents, verified supplier data (weight: 1.0)
- Silver Sources: Structured product metadata, curated FAQs (weight: 0.7)
- Bronze Sources: Chat logs, user reviews (weight: 0.3)
- Conflict Resolution: Higher-tier sources always override lower-tier when contradictions detected

Pillar 4: Observability & Audit

- Logging: Every inference logged with input hash, output, model version, safety scores

- Monitoring: Arize for real-time safety metric tracking and drift detection
- Audit Trail: Immutable log storage (S3 + Athena) for compliance investigations
- Alerting: PagerDuty integration for safety score anomalies (>2 std from baseline)

Results

Metric	Before	After	Method
Safety Gate Coverage	34%	100%	All production endpoints integrated
High-Risk Incidents	~12/month	1/month	Severity-weighted incident count
False Positive Rate	—	2.3%	Manual review of flagged queries
Validation Latency Overhead	—	8ms P95	End-to-end measurement
Compliance Audit Pass Rate	—	100%	External audit Q4 2024

System Design & Architecture
Modular Safety Layer integrated into LLMOps pipeline:
- Async Validation: Non-blocking for low-risk queries; blocking for flagged content
- Fallback Responses: Pre-approved safe responses for blocked queries
- A/B Testing: New safety models tested in shadow mode before promotion

Risks & Mitigations

Risk	Impact	Mitigation	Monitoring
Latency Bloat	Safety checks slow UX	SLMs + batching + parallel execution	P95 latency dashboard
False Positives	Harmless queries blocked	HITL review queue; threshold tuning	FP rate by category
Adversarial Bypass	Safety layer circumvented	Red team + encoding detection + multi-turn analysis	Canary token trigger rate
False Negatives	Harmful content passes	Layered detection; human audit of random sample	Weekly audit of 100 outputs
Model Drift	Safety model degrades over time	Continuous retraining on new examples	Safety score distribution tracking

---

Leadership & Strategic Impact

Beyond individual projects, I led strategic initiatives that shaped Viator's AI capabilities:

AI Strategy & Roadmap

- Landscape Analysis: Conducted comprehensive audit identifying strategic gaps in product discoverability and customer support
- Architecture Vision: Led transition to 'Search AI Lake' architecture, decoupling compute and storage for flexible model deployment
- Capability Building: Established cross-functional 'AI Readiness' programs educating non-technical stakeholders on LLM capabilities and risks

Team & Organizational Growth

- Scaling: Grew AI function from 0 to 12+ principals while maintaining quality bar
- Culture: Established 'Ablation-First' research culture—teams demonstrate diminishing returns on cost-performance curves before large-scale deployment
- Knowledge Sharing: Monthly AI Guild sessions sharing learnings across product teams

Aggregate Impact

Initiative	Metric	Impact
Search Personalization	NDCG@10	+4% (translates to ~$2M incremental annual revenue)
FAQ Automation	Ticket Deflection	34% reduction (~$2.4M annual savings)
Prompt Optimization	Inference Cost	18% reduction across classification workloads
Review Summarization	Token Usage	82% reduction in LLM costs for summaries
Safety Framework	Incidents	92% reduction in high-risk outputs

Cross-Portfolio Engineering Practices

The following practices are applied consistently across all projects:

A/B Testing Methodology

- Sample Size: Minimum detectable effect calculated pre-experiment; typical n > 500K sessions
- Duration: Minimum 2 weeks to capture weekly seasonality; 3 weeks for high-stakes changes
- Metrics: Primary metric (e.g., NDCG) + guardrail metrics (latency, error rate, revenue)
- Analysis: Bayesian analysis with 95% credible intervals; sequential testing for early stopping

Offline-Online Metric Alignment

Offline metrics often don't translate to online gains. Practices:

- Calibration Studies: Quarterly analysis of offline lift vs. online lift across past experiments
- Discount Factor: Apply 0.7x multiplier to offline gains when projecting online impact
- Hybrid Evaluation: Interleaving experiments for ranking models provide online signal without full traffic split

Model Monitoring & Drift Detection

- Feature Drift: KL divergence between training and serving feature distributions; alert at >0.1
- Prediction Drift: Monitor prediction distribution shift; alert at >5% mean shift
- Performance Monitoring: Delayed labels used to compute online metrics with 7-day lag
- Tooling: Arize for ML observability; custom Grafana dashboards for business metrics

Data Versioning & Reproducibility

- Dataset Versioning: DVC for training data; each model tagged with data version hash
- Experiment Tracking: MLflow for hyperparameters, metrics, and model artifacts
- Reproducibility: Docker images pinned with exact dependency versions; random seeds logged

Embedding Model Selection Framework: When selecting embedding models, evaluate:

Criterion	Benchmark	Threshold
Task Relevance	Downstream task performance (e.g., retrieval recall)	Top-2 on internal benchmark
Latency	P95 inference time	<20ms for online; <100ms for offline
Dimension/Cost	Storage and compute cost	Balance with quality needs
Language Coverage	Performance on non-English data	>90% of English performance

Failure Mode Analysis Template

Each project includes explicit failure mode documentation:

1. Identify top-5 failure modes during design phase
2. Implement detection mechanisms for each failure mode
3. Define automated response (alert, fallback, rollback)
4. Post-mortem template for production incidents
5. Quarterly review of failure mode coverage

Latency-Accuracy Tradeoff Documentation

All ML systems document their Pareto frontier:

- Model Variants: Test 3+ model sizes/architectures
- Pareto Chart: Plot accuracy vs. latency; identify knee points
- Operating Point: Document chosen tradeoff with business justification
- Fallback Tiers: Define degraded modes for latency spikes